

Lab 1 - SQL Injection

The application

You are testing **Meridian**, an internal service desk. Customers raise tickets, and support agents work them in queues. Start the stack and open the site:

```
docker compose -f lab/docker-compose.yml up --build
```

Then go to <http://127.0.0.1:5001> and sign in as an agent:

```
username: rmasri  
password: Agent1234!
```

The agent tools include an **advanced ticket search**. That is your target for this lab.

Goal

Somewhere in the search, what you type reaches the database as part of a SQL query. Prove that you can change what those queries do, and use that to read data the search was never meant to return.

How to work

Attack the running application as an outsider would: through the browser and through Burp, **without reading the source code**. Leave the code closed until the final step. Work every attack out yourself; there are no payloads in this sheet.

Proxy your traffic through Burp so you can capture the evidence you submit.

Objectives

1. Make the search return a ticket that your search term does not actually match.
2. Make the search return every ticket at once.
3. Read data the search was never designed to expose: the staff accounts and the password hash stored with each one.
4. Recover the administrator's stored password hash through a request whose response shows you no data at all. Do it one character at a time, and automate it.
5. Achieve the same result on a request whose response is identical every time, so you must infer each answer some other way. Explain what you measured.

Burp Community throttles Intruder to about one request per second, and its payload lists are a

Professional feature. A short script of your own will be faster for objectives 4 and 5.

Evidence to submit

For each objective, submit a Burp screenshot showing the request you sent and the response that proves you reached it:

- objective 1: the unexpected ticket returned
- objective 2: every ticket returned
- objective 3: the staff usernames and password hashes returned
- objective 4: the recovered hash, and the script you used
- objective 5: your evidence, with the measurement that proves it

Remediation

Only now open the source. Find every place where your input reached the SQL query, and fix it. Re-run every attack above and confirm that each one now fails. Submit the before and after for one of them, and name the change you made.